

CS361 Project1 Report

Beckett Dunlavy and Tomas Salaz

Design

The Code implements the BST structure using the Singleton Pattern and uses Inheritance and Polymorphism.

Design : $BT \rightarrow Nil \mid Node(num, BT, BT)$

The BT implements the design described in the *Project1.docx* where

BT $\rightarrow$ Nil represents an empty Tree

BT $\rightarrow$ Node(num, BT, BT) represents a Tree Node with *num* value, and Right and Left Subtree's, *BT*.

Classes

The Binary Tree Abstract Class defines the interfaces for the BT, those being *height()*, *isBST()*, *delete(value)*, and *makeBST(values [])*.

Nil Inner Class implements the singleton empty Tree, returns *height* = -1 is the only important attribute of the Nil class.

Node Inner Class represent a non-empty BT containing *num* as the data held in the node. Also holds references to the Left and Right subtrees.

Node is where the interfaces from above are implemented. This makes it easier to do operations on subtrees and use recursion.

Run Time Analysis

height(): Each Node does $O(1)$ work to calculate the height and recurses on both children so we get $T(n) = T(nl) + T(nr) + O(1)$, summed over all nodes, $T(n) = \sum O(1) = O(n)$
So the Runtime for *height()* is $O(n)$

isBST(): Checks whether the tree satisfies BST ordering rules using a recursive helper that tracks a **minimum** and **maximum** allowed value. Each node must fall strictly within these bounds. When recursing into the left subtree, the current node's value becomes the new maximum; when recursing into the right subtree, it becomes the new minimum. If any node violates its range, the tree is not a BST. Since each node is visited once, the runtime is $O(n)$.

makeBST(int[] values) with insert(): We make a BST by inserting values into the subtrees one by one. Let C_k be the cost to insert the k -th value into a tree with size $k - 1$. Since there is no rebalancing, insertion run time is based on the height of the subtree, called H_{k-1} .

Thus $C_k = H_{k-1}$

For the Worst Case where each insertion follows the same path so

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2)$$

For the Best Case the tree remains balanced, so each insertion occurs along a path proportional to the height of a complete BST of size

$k-1$, which is approximately $\log_2(k-1)$. Thus the cost of inserting the k -th element becomes:

$$C_k = \Theta(\log k)$$

Summing over all insertions gives:

$$T(n) = \sum_{k=1}^n \Theta(\log k) = \Theta(n \log n)$$

Therefore, the running time to construct a BST by repeated insertion varies based on structure:

- **Best Case (balanced tree):** $\Theta(n \log n)$
- **Worst Case (degenerate list-shaped tree):** $\Theta(n^2)$

delete(value): The Deletion implementation starts by searching for the key. This operation takes at most $\theta(h)$ time.

Once the node is found there are 3, cases:

-Key is a Leaf, has $O(1)$ time

-Key has 1 child, has $O(1)$ time since we return that child

-Key had 2 Children, need to calculate Run Time to find successor. To find the Successor, we walk the Left Side of the Right Subtree, this is based on the height called h_k . When we add all the terms for searching, finding successor and replacement we get

$$T(n) = \theta(h) + \theta(h_k) + O(1)$$

For the Worst Case, where a tree only has left subtrees, we are bound by $\theta(h)$ which in this case is also $\theta(n)$

For the Best Case, where a Tree is balanced. We can alter the equation to where $T(n) = C1 * h + C2 * h + O(1)$ which can be reduced to $T(n) = O(\log n)$